

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Web Client and Backend Development Framework.

Reporte de Configuración SMTP y Envío de Correo Electrónico con Gmail y
Spring Boot

Profesor: M. en C. José Asunción Enríquez Zárate

Alumno: Luis David Martínez Martínez

ld.martinez.117@gmail.com

7CM3

22 de Junio, 2026

Contents

1	Introducción	1
2	Conceptos	2
2.1	<i>Protocolo SMTP y puertos seguros</i>	2
2.2	<i>JavaMailSender</i>	2
2.3	<i>Google App Passwords (Contraseñas de Aplicación)</i>	2
3	Desarrollo	3
3.1	<i>Paso 1: Configuración de la Cuenta de Google</i>	3
3.2	<i>Paso 2: Dependencias del Proyecto</i>	3
3.3	<i>Paso 3: Propiedades del Servidor SMTP</i>	3
3.4	<i>Paso 4: Implementación del Servicio de Correo</i>	3
4	Resultados	4
5	Conclusión	6
6	Referencias Bibliográficas	7

List of Figures

1	Petición de prueba para el envío de correo desde la interfaz de Swagger UI.	4
2	Evidencia de correo electrónico recibido de forma exitosa en el cliente de correo.	5

List of Tables

1 Evaluación de Criterios Práctica 2 6

1 Introducción

El soporte para envío de notificaciones automáticas y alertas asíncronas es un requerimiento ineludible en el desarrollo de plataformas empresariales web. A través de este canal de comunicación, los sistemas notifican registros de cuenta, facturas digitales, o notificaciones sobre el estado de reservaciones.

En el ecosistema Java, Spring Boot proporciona el starter de correo (**spring-boot-starter-mail**) para simplificar la integración de clientes SMTP. Sin embargo, para usar servidores de producción como Gmail en aplicaciones locales, se deben aplicar políticas estrictas de seguridad (como OAuth o contraseñas de aplicación) para evitar brechas de seguridad. En este reporte se detalla el flujo de configuración segura para interactuar con los servidores de Google y la implementación de un API simple en Spring Boot para realizar el envío.

2 Conceptos

2.1 *Protocolo SMTP y puertos seguros*

El protocolo SMTP (Simple Mail Transfer Protocol) es el estándar de red utilizado para el envío de correo. Para asegurar la transmisión de contraseñas y datos del correo, se utiliza STARTTLS sobre el puerto 587 o bien cifrado SSL directo sobre el puerto 465.

2.2 *JavaMailSender*

Es la interfaz principal provista por Spring Framework que unifica la creación y transmisión de mensajes de correo simple (`SimpleMailMessage`) o complejos en formato HTML (`MimeMessage`).

2.3 *Google App Passwords (Contraseñas de Aplicación)*

Es un mecanismo provisto por Google para cuentas que tienen activada la autenticación en dos pasos (2FA). Consiste en generar un código de acceso único de 16 caracteres para autorizar que aplicaciones específicas de terceros interactúen con el servidor de correo emisor en nombre del usuario, omitiendo la contraseña principal.

3 Desarrollo

3.1 Paso 1: Configuración de la Cuenta de Google

Para permitir que Spring Boot use Gmail, el usuario debe realizar las siguientes acciones en la consola de seguridad de su cuenta Google:

1. Activar la verificación en dos pasos (2FA).
2. Generar una contraseña de aplicación bajo la sección “Seguridad”.
3. Copiar el token de 16 caracteres autogenerado.

3.2 Paso 2: Dependencias del Proyecto

En el archivo `pom.xml` del backend se incorporó la librería de mensajería:

```
1 <dependency>
2     <groupId>org.springframework.boot</groupId>
3     <artifactId>spring-boot-starter-mail</artifactId>
4 </dependency>
```

3.3 Paso 3: Propiedades del Servidor SMTP

En el archivo de configuración `application.properties` se agregaron los parámetros de red correspondientes a Google Mail:

```
1 spring.mail.host=smtp.gmail.com
2 spring.mail.port=587
3 spring.mail.username=${MAIL_USERNAME}
4 spring.mail.password=${MAIL_PASSWORD}
5 spring.mail.properties.mail.smtp.auth=true
6 spring.mail.properties.mail.smtp.starttls.enable=true
```

Las variables de entorno son cargadas dinámicamente desde el archivo de secretos locales `.env`.

3.4 Paso 4: Implementación del Servicio de Correo

Se estructuró la interfaz del servicio y su clase concreta de implementación:

```
1 @Service
2 @RequiredArgsConstructor
3 public class MailServiceImpl implements MailService {
4     private final JavaMailSender mailSender;
5
6     @Override
7     public void sendEmail(String to, String subject, String body) {
8         try {
9             SimpleMailMessage message = new SimpleMailMessage();
10             message.setFrom("no-reply@reservaciones.com");
11             message.setTo(to);
12             message.setSubject(subject);
13             message.setText(body);
14             mailSender.send(message);
15         } catch (Exception e) {
16             System.err.println("Fallo al enviar correo: " + e.getMessage());
17         }
18     }
19 }
```

4 Resultados

Para constatar el funcionamiento del correo, se expuso un controlador dedicado:

```
1 @RestController
2 @RequestMapping("/api/v1/mail")
3 @RequiredArgsConstructor
4 public class MailController {
5     private final MailService mailService;
6
7     @PostMapping("/send-test")
8     public ResponseEntity<String> sendTestEmail(
9         @RequestParam String to,
10        @RequestParam String subject,
11        @RequestParam String body) {
12        mailService.sendEmail(to, subject, body);
13        return ResponseEntity.ok("Correo enviado con éxito.");
14    }
15 }
```

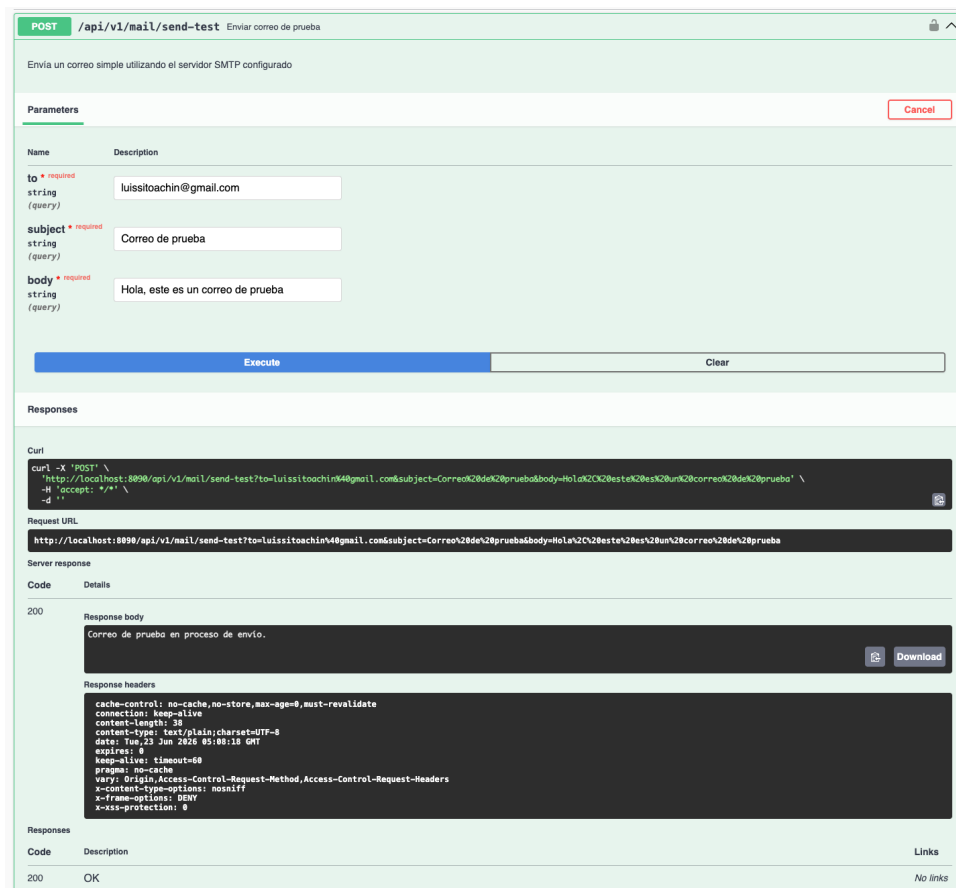


Figure 1: Petición de prueba para el envío de correo desde la interfaz de Swagger UI.

El destinatario del correo recibe una notificación en su bandeja de entrada detallando el cuerpo del mensaje redactado, confirmando la correcta comunicación con los servidores SMTP de Google. Además de poder observarse en la parte superior derecha que la cuenta de correo revisada no es la misma que la perteneciente al emisor del correo.

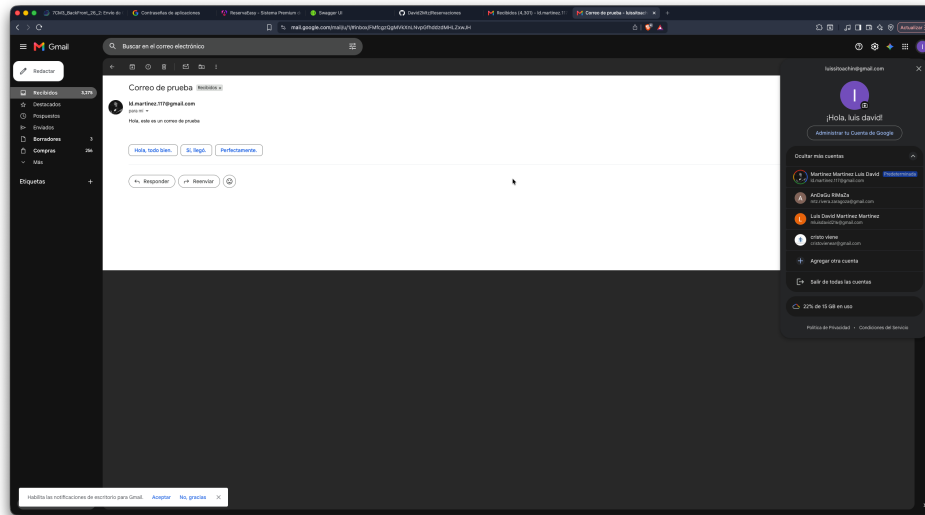


Figure 2: Evidencia de correo electrónico recibido de forma exitosa en el cliente de correo.

5 Conclusión

La integración de SMTP mediante el starter de Spring Boot facilita el desarrollo de notificaciones por correo. Se ha comprobado que el esquema de contraseñas de aplicación permite utilizar servidores de producción como Gmail en entornos de pruebas de forma muy sencilla, manteniendo los estándares de seguridad de datos exigidos hoy en día por la industria.

Criterio	Puntos	Realizado
Portada	Sin	100%
Introducción	2	2
Conceptos	2	2
Desarrollo	2	2
Resultados	2	2
Conclusiones	1	1
Referencias	1	1
Total	10	10

Table 1: Evaluación de Criterios Práctica 2

Luis David Martínez Martínez

6 Referencias Bibliográficas

References

[Google Security, 2026] Google Help. *Sign in using App Passwords*. Google Accounts Support. Disponible en:
<https://support.google.com/accounts/answer/185833>

[Spring Mail, 2026] Spring Projects. *Email integration in Spring Framework*. VMware. Disponible en:
<https://docs.spring.io/spring-framework/docs/current/reference/html/integration.html#mail>